Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет

по лабораторной работе № 5

на тему «Применение механизмов безопасности и обработки запросов в GraphQL API»

Выполнил: Летко А.С.
студент группы 377901

Проверила: Сильванович Ю.В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Цель работы – встроить в разработанную ранее платёжную систему механизмы аутентификации и авторизации, защитить GraphQL API и ограничить выполнение операций с учётом роли пользователя и принадлежности данных.

2 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

1.1 Исходная система и архитектура решения

В качестве основы использована платёжная система из предыдущей лабораторной работы. Она содержит GraphQL Gateway и модули предметной области: пользователей, продавцов, платежей, способов оплаты, возвратов, транзакций и аналитики. Данные хранятся в PostgreSQL. Бизнес-модули подключены к приложению шлюза как зависимости Maven и вызываются через интерфейс ServiceClient и реализацию MockServiceClient.

Для безопасности добавлены отдельное приложение Auth Service и Redis. Все компоненты запускаются через Docker Compose: база данных db, хранилище redis, auth-service и gateway. Из перечисленных приложений внешний HTTP-порт предоставляет шлюз; Auth Service и Redis взаимодействуют с ним внутри сети Compose. PostgreSQL также опубликован на локальном интерфейсе для работы с базой.

Шлюз принимает GraphQL-запрос, а Auth Service выполняет регистрацию, проверку пароля, выдачу и проверку JWT и управление сессиями. После проверки шлюз определяет права по роли и владельцу данных. Существующие бизнес-модули не получают JWT и не проверяют его повторно. При этом роль и идентификатор пользователя используются непосредственно в контроллере шлюза, а не передаются отдельным параметром в PaymentService; это ограничение текущей реализации.

Взаимодействие компонентов при защищённом запросе состоит из нескольких последовательных шагов. Браузер направляет документ GraphQL на адрес шлюза; шлюз извлекает заголовок Authorization и обращается к внутреннему HTTP-методу проверки Auth Service. Сервис проверяет токен и сессию и возвращает сведения о пользователе. Только после этого контроллер решает, разрешена ли операция, и вызывает соответствующий метод ServiceClient.

Такое разделение позволяет хранить данные учётных записей и логику проверки пароля в одном компоненте. Шлюзу не требуется знать пароль пользователя или хранить сессии в собственной памяти. С другой стороны, доступность защищённых операций зависит от работоспособности Auth Service и Redis: если проверку выполнить невозможно, предоставлять доступ на основании одного клиентского утверждения нельзя.

Docker Compose задаёт единое окружение запуска. Контейнер PostgreSQL инициализируется схемами предметной области, Redis предоставляет быстрое хранилище сессий, а оба Java-приложения собираются из одного Maven-проекта. Зависимости запуска учитывают готовность базы и Redis. Персистентный том database сохраняет данные базы между обычными остановками проекта.

1.2 Пользователи, роли и регистрация

Для входа используются адрес электронной почты и пароль. Пользователи и их роли хранятся в таблице identity.users, а учётные данные – в таблице auth.credentials. Пароль сохраняется в виде BCrypt-хеша, а не открытым текстом. Адрес нормализуется к нижнему регистру; ограничение уникальности защищает от повторной регистрации одного адреса.

Регистрация доступна без токена через мутацию register. Сервис проверяет обязательные поля, формат адреса и длину пароля, создаёт идентификатор и записывает данные. Новому пользователю назначается только роль CLIENT: выбрать ADMIN или MERCHANT в запросе регистрации нельзя. Для учебной демонстрации предусмотрены клиент anna@example.com, продавец pavel@example.com и администратор admin@example.com; их начальные учётные данные описаны в README проекта.

Записи пользователя и учётных данных создаются в транзакции базы. Если сохранение не удаётся, операция не должна оставить половину новой учётной записи. BCrypt использует соль и вычислительно затратное преобразование, поэтому одинаковые пароли не обязаны иметь одинаковые хеши. При следующем входе сравнение выполняется специальной функцией проверки BCrypt.

Роли хранятся отдельно от содержимого клиентского запроса. Результат проверки пользователя получается из базы, поэтому передача клиентом произвольного текста роли в GraphQL не изменяет его права. Начальные пользователи и администратор создаются для демонстрации; для реального развёртывания их пароли и секрет подписи следует заменить.

На рисунке 1.1 показана регистрация без авторизации: поле токена пустое, а ответ содержит созданный идентификатор, имя, адрес и роль CLIENT. Для повторного выполнения необходимо выбрать ещё не зарегистрированный адрес. Соответствующий запрос приведён в листинге 1.1.

Листинг 1.1 - Регистрация пользователя

```
mutation {
  register(name: "Student Report", email: "report5c@example.com",
           password: "Password123!") { id name email role }
}
```

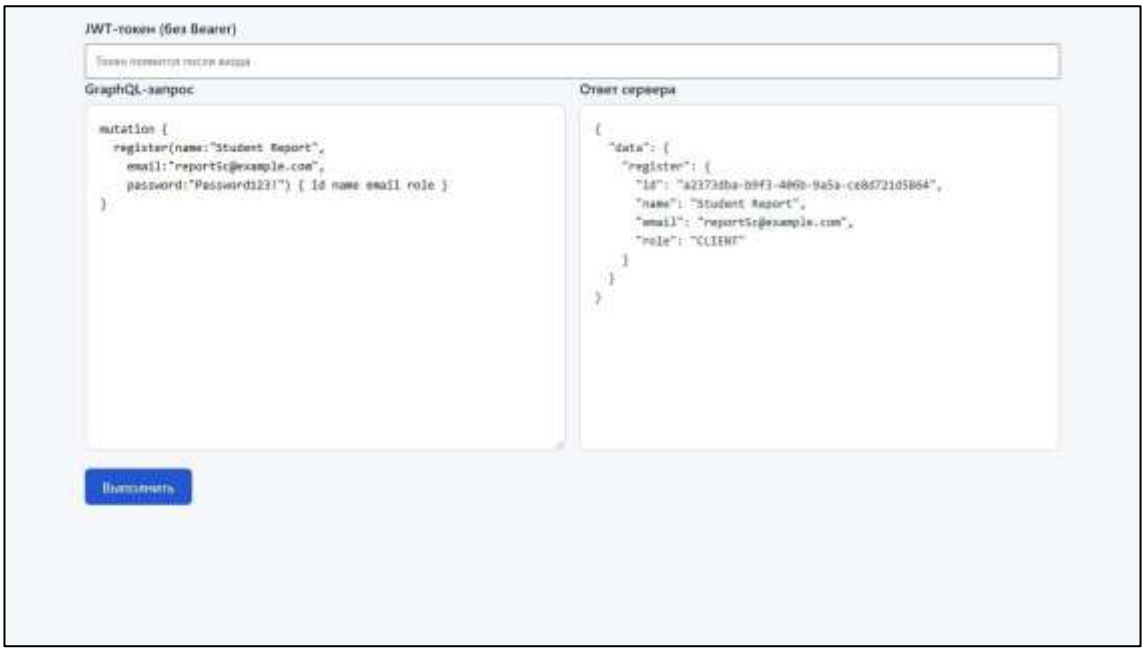


Рисунок 1.1 - Регистрация пользователя

1.3 Аутентификация и выдача JWT

Мутация login передаёт учётные данные из шлюза в Auth Service. Сервис находит запись, сравнивает пароль с BCrypt-хешем и при успехе создаёт отдельный идентификатор сессии. В подписанный JWT включаются идентификатор пользователя, идентификатор сессии, время выдачи и время окончания действия. Клиент получает токен, время жизни и сведения о пользователе.

При проверке работающего приложения вход учебного клиента завершился успешно. На рисунке 1.2 показаны срок действия 3600 секунд и роль CLIENT; поле token в этом демонстрационном запросе опущено, чтобы не включать действующий токен в отчёт.

Полный ответ с полем token и его автоматическую подстановку в поле JWT-токена дополнительно показывает рисунок 1.3. Показанная на снимке сессия была завершена после проверки. Соответствующий запрос приведён в листинге 1.2.

Листинг 1.2 - Аутентификация пользователя

```
mutation {
  login(email: "anna@example.com", password: "Lab12345!") {
    expiresIn user { id name role }
  }
}
```

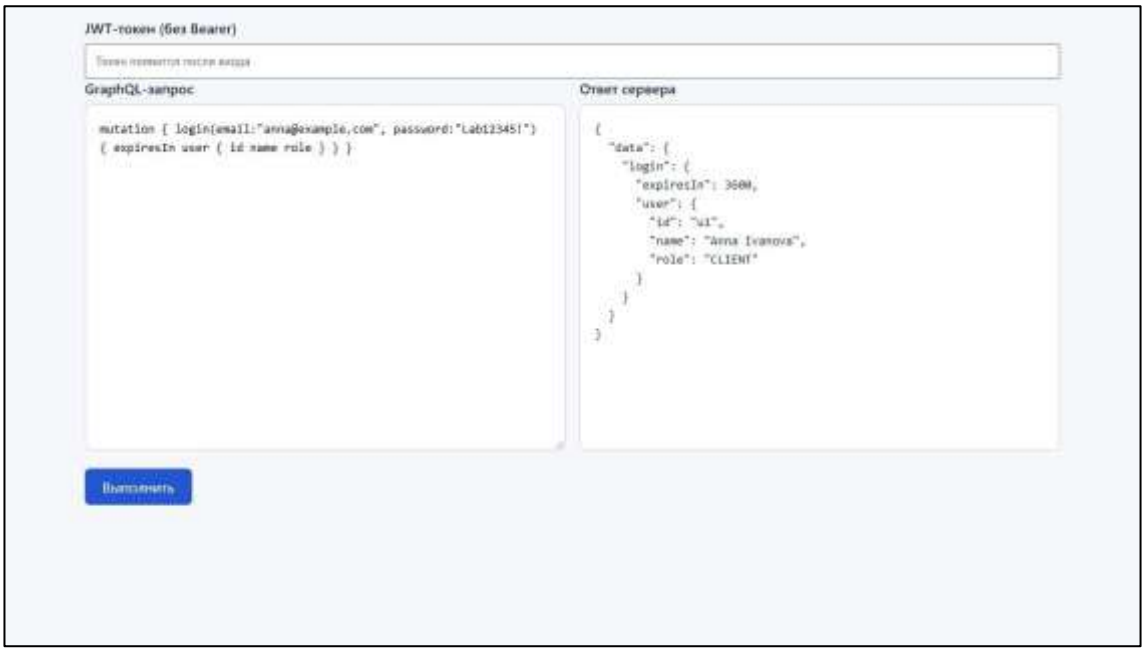


Рисунок 1.2 - Успешная аутентификация клиента

При выполнении мутации login с полем token сервер возвращает подписанный JWT в JSON-ответе. На рисунке 1.3 токен виден в ответе справа и автоматически скопирован страницей в поле JWT-токена над запросом. Поле expiresIn показывает срок действия, а объект user содержит идентификатор и роль вошедшего пользователя.

Далее страница отправляет значение поля в заголовке Authorization: Bearer при выполнении запросов. Токен вводится без слова Bearer: его добавляет клиентский код страницы. Для входа с другой учётной записью достаточно выполнить новую мутацию login; при logout страница очищает поле, а сервер удаляет соответствующую сессию. Соответствующий запрос приведён в листинге 1.3.

Листинг 1.3 - Получение JWT-токена

```
mutation {
  login(email: "anna@example.com", password: "Lab12345!") {
    token expiresIn user { id name role }
  }
}
```



Рисунок 1.3 - Выдача JWT и поле токена

При проверке с неверным паролем сервер вернул ошибку UNAUTHORIZED и не выдал токен. Результат этого сценария показан на рисунке 1.4.

Аутентификация и авторизация решают разные задачи. Первая отвечает на вопрос, кто обратился к системе, а вторая – может ли установленный пользователь выполнить конкретное действие. Успешный вход не означает, что пользователь автоматически получает права администратора или доступ к данным другого клиента.

JWT содержит служебные утверждения, в том числе sub с идентификатором пользователя, jti с идентификатором сессии, iat со временем выдачи и exp со сроком окончания. Сервер проверяет подпись и срок действия. Содержимое токена нельзя считать секретным: важна проверяемая подпись, а не сокрытие данных внутри него. Соответствующий запрос приведён в листинге 1.4.

Листинг 1.4 - Вход с неверным паролем

```
mutation { login(email: "anna@example.com",
  password: "wrongpassword") { token } }
```

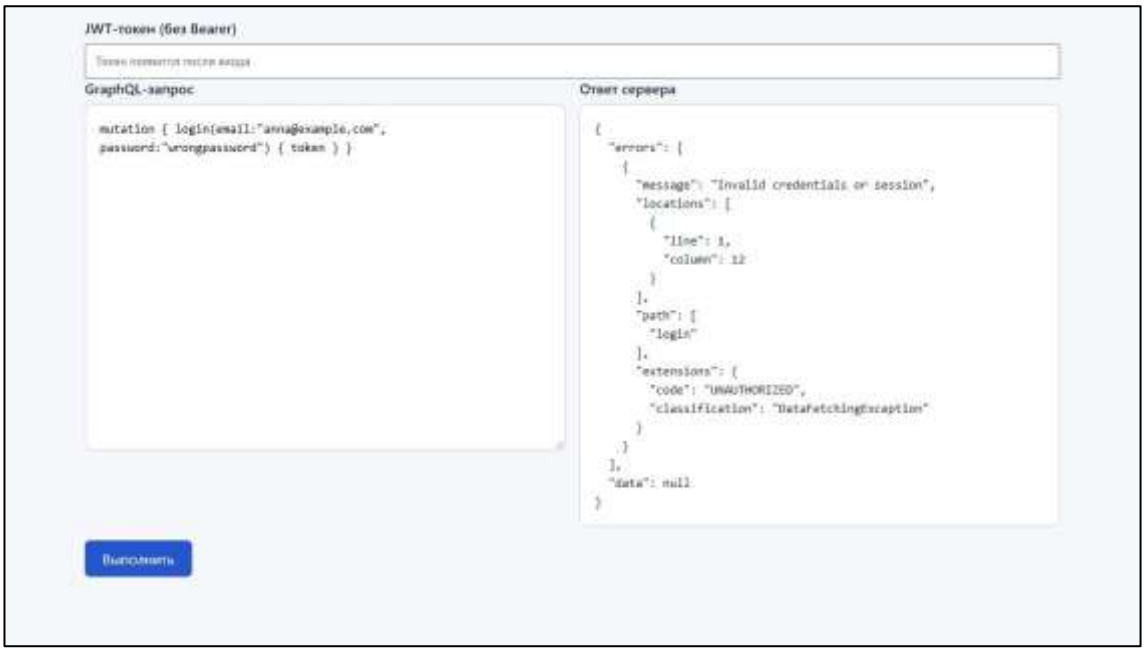


Рисунок 1.4 - Отказ при неверном пароле

1.4 Активная сессия в Redis

После успешного входа Auth Service создаёт запись session:<идентификатор сессии>, значением которой является идентификатор пользователя. Для записи задаётся время жизни TTL; стандартное значение составляет 3600 секунд и задаётся параметром SESSION_SECONDS. Одному пользователю могут соответствовать разные сессии, поскольку при каждом входе создаётся новый идентификатор.

Действующей подписи JWT недостаточно: сервис также считывает запись из Redis и сравнивает её значение с идентификатором пользователя из токена. Отсутствующая или истёкшая запись означает, что запрос не авторизован. Роль считывается из PostgreSQL при проверке, поэтому для решения о доступе используется актуальная роль.

Для иллюстрации хранения сессии на рисунке 1.5 можно показать вывод Redis после входа. Команды KEYS и TTL ниже предназначены только для локальной учебной среды; вместо <ключ> необходимо подставить полученный ключ. Значение TTL уменьшается со временем. Соответствующие команды приведены в листинге 1.5.

Листинг 1.5 - Проверка активной сессии в Redis

```
docker compose exec redis redis-cli KEYS "session:*"
docker compose exec redis redis-cli TTL "<ключ>"
```

```
PS D:\Study\term-7\ИИСиК\lab-work-5> docker compose exec redis redis-cli KEYS "session:*"
1) "session:61c4f22d-379c-47e0-bbfd-5f0c3e936dc1"
2) "session:a2279bd9-1da5-47d6-899e-3c672a7b18d6"
3) "session:1bdeec9d-584a-4870-b57d-cc6931702a61"
4) "session:a02121ab-2c47-4176-86ad-904f0f16553d"
5) "session:149480ad-de7c-41b5-88a5-5802f8c69451"
6) "session:566646f4-57bf-4e73-94b1-7a5e53d7f391"
7) "session:1a1f5394-eacd-4089-b093-2f2988258924"
PS D:\Study\term-7\ИИСиК\lab-work-5> ^C
PS D:\Study\term-7\ИИСиК\lab-work-5> docker compose exec redis redis-cli TTL "session:1a1f5394-eacd-4089-b093-2f2988258924"
(integer) 2128
```

Рисунок 1.5 - Активная сессия и её время жизни в Redis

Мутация logout удаляет только текущую запись сессии. После успешного выхода прежний JWT был повторно передан в защищённом запросе; сервер ответил Invalid token or inactive session и UNAUTHORIZED. Это подтверждает проверку сессии независимо от наличия ранее выданного токена (рисунок 1.6).

Время жизни JWT и время жизни записи Redis задаются одним значением. Запись Redis автоматически исчезает после TTL без отдельного задания очистки. Досрочное удаление записи при выходе позволяет завершить доступ раньше окончания срока действия токена. Запуск новой сессии выдаёт новый идентификатор, поэтому отзыв одной сессии не описывается как обязательный выход из всех остальных. Соответствующий запрос приведён в листинге 1.6.

Листинг 1.6 - Завершение сессии и повторный запрос

```
mutation { logout }

# Затем повторить защищённый запрос со старым токеном:
query { payments { id status } }
```



Рисунок 1.6 - Прекращение сессии и повторный запрос

1.5 Защита GraphQL API

В браузерной странице пользователь вводит запрос слева и получает JSON-ответ справа. После входа токен подставляется в поле и отправляется в заголовке Authorization со схемой Bearer. Перед обращением к защищённому корневому полю шлюз передаёт токен в Auth Service. Тот проверяет подпись и срок JWT, сессию в Redis и наличие пользователя; при ошибке защищённая операция не выполняется.

Запрос payments без токена был проверен на работающем приложении. В ответе получено сообщение Authentication required с кодом UNAUTHORIZED, что следует показать на рисунке 1.7. Соответствующий запрос приведён в листинге 1.7.

Листинг 1.7 - Запрос платежей без токена

```
query { payments { id amount status } }
```

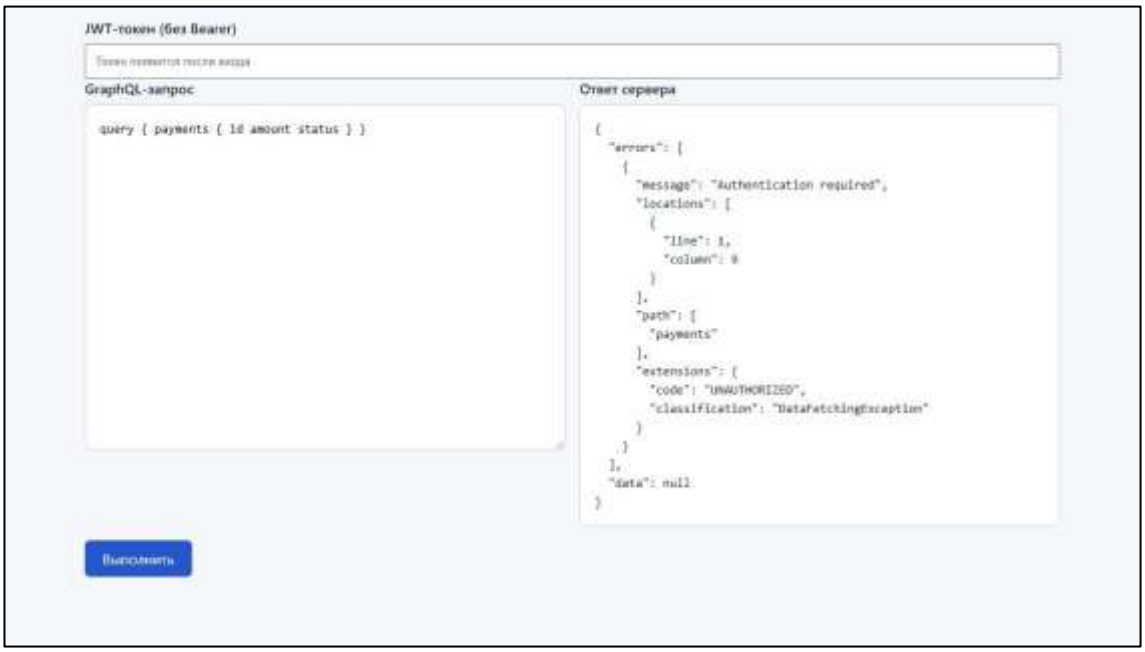


Рисунок 1.7 - Отказ в доступе без токена

После успешного входа как anna@example.com тот же защищённый запрос вернул платежи клиента u1. В проверенном ответе присутствовали p1 со статусом SUCCESS и p2 со статусом CREATED. Запрос и результат следует показать на рисунке 1.8.

Для списка платежей контроллер создаёт копию фильтра и добавляет clientId текущего клиента. Если клиент явно запрашивает чужой clientId, операция отклоняется. Для продавца аналогично учитывается принадлежность merchantId. Благодаря этому фильтрация применяется до

формирования списка и агрегированного результата, а не только при отображении на странице.

Вложенные поля платежа также требуют внимания: при разрешённом доступе к платежу можно запросить связанного продавца, возвраты или транзакции. Доступ к платёжному методу дополнительно ограничен владельцем или администратором. Политика распространяется на отдельные корневые поля Query и Mutation, а не только на кнопки пользовательского интерфейса. Соответствующий запрос приведён в листинге 1.8.

Листинг 1.8 - Получение платежей клиента

```
query {
  payments { id clientId merchantId amount currency status }
}
```



Рисунок 1.8 - Получение доступных платежей клиента

1.6 Роли и разграничение доступа

Реализованы роли CLIENT, MERCHANT и ADMIN. Клиент получает доступ к собственным платежам, связанным транзакциям и возвратам, а также к своим способам оплаты. Для списка платежей и сводки шлюз принудительно учитывает идентификатор клиента. Клиент может создать платёж от своего имени и отменить свой платёж в допустимом состоянии.

Продавец получает доступ к платежам, транзакциям и возвратам своего магазина, а также к его аналитике и отчётам. Он может подтвердить платёж принадлежащего ему магазина. Администратор имеет доступ ко всем предусмотренным данным и операциям. Запрос user по идентификатору

ограничен самим пользователем или администратором; вложенный способ оплаты доступен владельцу и администратору.

Попытка клиента подтвердить платёж p2 была проверена на работающей системе: операция completePayment вернула Access denied и код FORBIDDEN. Таким образом, наличие корректного токена само по себе не даёт права на любую мутацию. Результат следует представить на рисунке 1.9.

Для одиночного платежа проверяется его принадлежность клиенту или магазину. Для связанных транзакций и возвратов система сначала определяет платёж, к которому относится запись, и затем применяет к нему то же правило доступа. Такое решение исключает ситуацию, когда общий список связанных объектов показывает данные по недоступному платежу.

Код UNAUTHORIZED указывает на отсутствие или недействительность подтверждения личности, в том числе на отсутствие активной сессии. Код FORBIDDEN означает, что пользователь установлен, но требуемое действие ему не разрешено. Различие видно на результатах проверок: без токена доступ отклоняется как UNAUTHORIZED, а подтверждение платежа клиентом – как FORBIDDEN. Соответствующий запрос приведён в листинге 1.9.

Листинг 1.9 - Попытка подтверждения платежа клиентом

```
mutation { completePayment(id: "p2") { id status } }
```

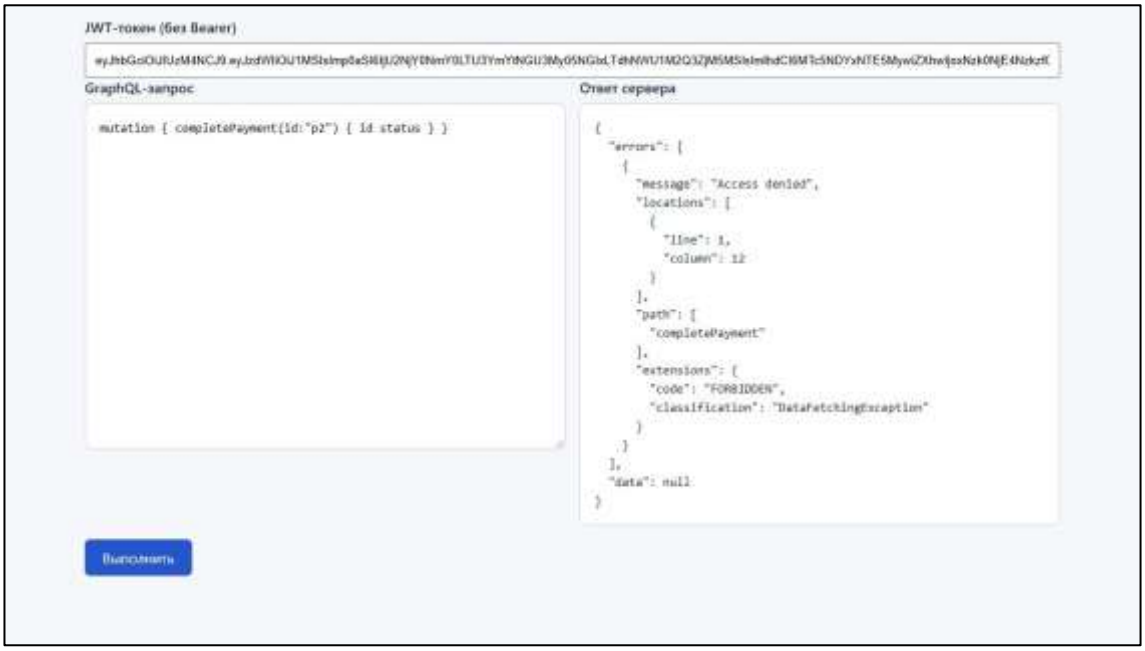


Рисунок 1.9 - Запрет подтверждения платежа клиентом

Вход как продавец pavel@example.com и запрос отчёта магазина m1 также проверены. На рисунке 1.10 показаны две записи отчёта с количеством платежей, суммой и выручкой. При запросе чужого магазина проверка

принадлежности должна отказать в доступе. Соответствующий запрос приведён в листинге 1.10.

Листинг 1.10 - Запрос отчёта продавца

```
query {
  reports(merchantId: "m1",
    period: {from: "2026-01-01", to: "2026-12-31"}) {
    merchantId paymentCount totalAmount revenue
  }
}
```

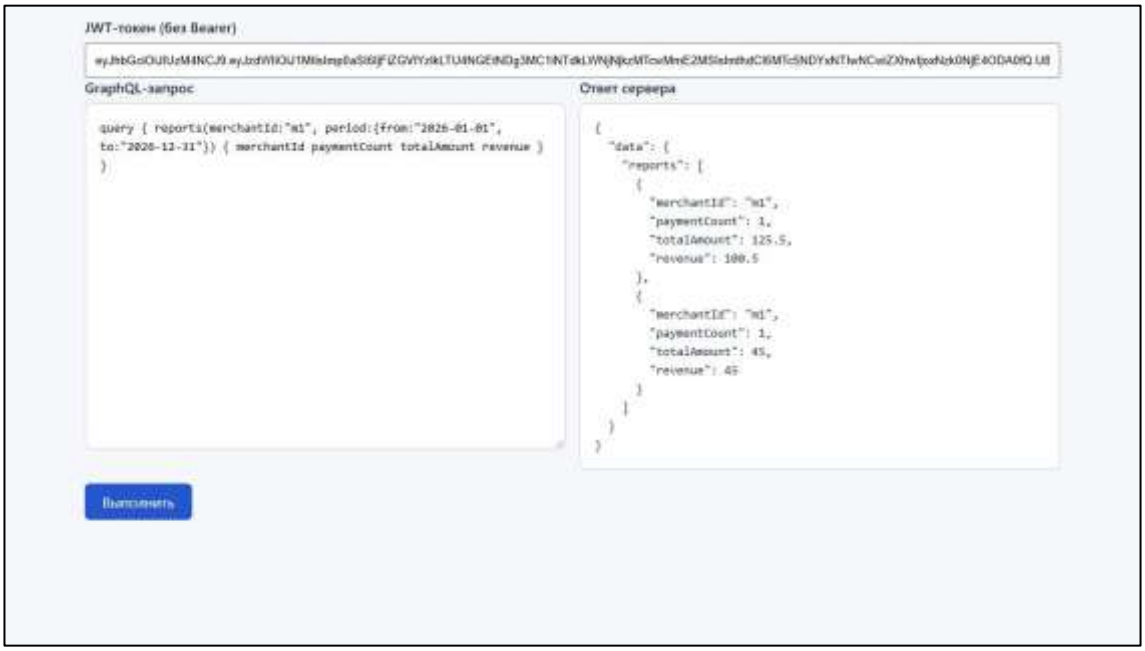


Рисунок 1.10 - Доступ продавца к отчёту своего магазина

1.7 Сохранение бизнес-правил

Проверка роли и принадлежности платежа выполняется в шлюзе до вызова метода изменения данных. Далее бизнес-логика предыдущей лабораторной работы проверяет существование клиента и продавца, активность способа оплаты и его принадлежность клиенту, допустимость суммы и валюты. Платёж создаётся со статусом CREATED.

Подтверждение и отмена переводят платёж соответственно в SUCCESS или CANCELLED только из состояния CREATED. Условие текущего статуса включено в SQL-обновление, поэтому недопустимый повторный переход не должен изменить запись. Ошибки входных данных и бизнес-правил возвращаются через GraphQL, а ошибки безопасности дополнительно содержат коды UNAUTHORIZED или FORBIDDEN.

При создании платежа роль и идентификатор пользователя проверяются до вызова метода create. Затем PaymentService выполняет проверки предметной области: сумма должна быть положительной, иметь не более двух

десятичных знаков и помещаться в тип базы данных. Код валюты должен состоять из трёх букв; способ оплаты должен принадлежать выбранному клиенту и иметь активный статус.

При изменении состояния условие WHERE учитывает прежнее значение CREATED. Если строка не обновлена, сервис различает отсутствие платежа и попытку изменить уже завершённый или отменённый платёж. Таким образом, авторизация и проверка допустимости операции решают разные задачи и обе выполняются до успешного изменения состояния.

Агрегированный запрос paymentSummary использует фильтр поиска и возвращает количество и сумму платежей. Перед вызовом его фильтр ограничивается текущим пользователем или магазином. Суммирование платежей с различными валютами без уточнения валюты не допускается, поскольку такая сумма не имела бы корректного финансового смысла.

1.8 Запуск и проверка

Для запуска требуется работающий Docker Desktop. В корневом каталоге проекта выполняется команда docker compose up --build -d. После запуска веб-страница GraphQL доступна по адресу http://localhost:18080/. По завершении работы контейнеры можно остановить командой docker compose down; данные PostgreSQL сохраняются в именованном томе.

При проверке приложения выполнены регистрация, вход клиента и продавца, запрос платежей и отчёта, отказ в подтверждении платежа клиентом, запрос без токена, вход с неверным паролем и повторное использование токена после выхода. Непосредственное наблюдение истечения TTL и состояния контейнеров оставлено для дополнительной демонстрации на рисунках 1.5 и 1.11.

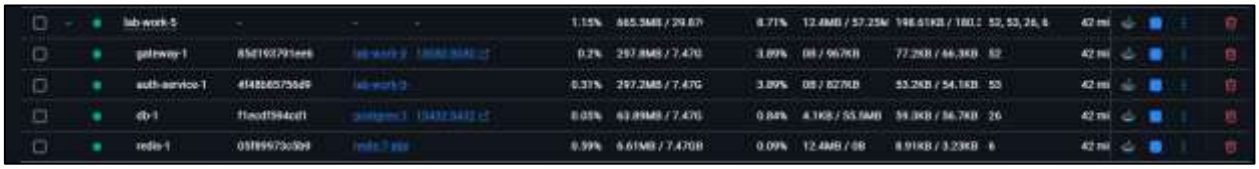


Рисунок 1.11 - Контейнеры платёжной системы в Docker Desktop

3 ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы платёжная система была расширена отдельным сервисом аутентификации, JWT и хранилищем активных сессий Redis. Для операций GraphQL добавлено разграничение по ролям и принадлежности данных, а пароли пользователей сохраняются в виде BCrypt-хешей.

На работающем приложении проверены регистрация, вход, получение собственных платежей и отчёта продавца, а также отказы при отсутствии

токена, неверном пароле, недостаточных правах и после выхода. Для дальнейшего развития следует передавать проверенный контекст пользователя непосредственно в бизнес-сервисы и дополнительно проверить естественное истечение TTL и остальные операции каждой роли.